

Practical: 2	Implementation and Time analysis of linear and binary search algorithm
05/08/2026	

Aim: To Implement Linear search and binary search and Time analysis their algorithms.

Linear search

Algorithm:

① Input :- Array element and search element
output :- position of key element
step ① :- start
step ② :- take the key element to search
step ③ :- set found = 0
step ④ :- for $i = 0$ to $i < \text{size}$ then do
step ⑤ :- if $\text{arr}[i] == \text{key}$ then do
step ⑥ :- set found = 1
step ⑦ :- Then break the loop
step ⑧ :- end for
step ⑨ :- if $i == \text{size}$ then do
step ⑩ :- element not found
step ⑪ :- stop

Program:

```
#include <iostream>

using namespace std;

int main() {
    int n;
    cout << "Enter the number of elements: ";
    cin >> n;

    int arr[n];    cout << "Enter " << n <<
" elements: ";    for (int i =
0; i < n; i++) {
    cin >> arr[i];
    }

    int target;    cout << "Enter the element
to search: ";
    cin >> target;

    int index = -1;    for (int i
= 0; i < n; i++) {        if (arr[i]
== target) {            index = i;
break;
        }
    }

    if (index != -1) {        cout << "Element found at index: "
<< index << endl;
    } else {        cout << "Element not
found" << endl;
    }

    return 0;
}
```

Output:

```
Enter the number of elements: 5
Enter 5 elements: 66
32
55
45
6
Enter the element to search: 6
Element found at index: 4
```

Time Complexity:

```
→ for(i=0; i<n; i++) { → n
    if (arr[i] == key) { → n
        cout << "element found"; → 1
        found = 1; → 1
        break; → 1
    }
}
T(n) = 7n + 8
T(n) = n
```

Average case

- element found in middle

Ex arr = {10, 20, 30, 40, 50}

key = 30

30 found at 2 index

↳ $n/2$

$$T(n) = n/2$$

$$T(n) = O(n)$$

Worst case

Element found at last

Ex arr = {10, 20, 30, 40, 50}

n comparison's of key

$$T(n) = n$$

$$T(n) = O(n)$$

Best case - $O(1)$

Ex - arr = {10, 20, 30, 40, 50}

key = 10

Then only 1 time comparison

$$T(n) = O(1)$$

Conclusion:

Hence From this practical of linear search, the algorithm and programs have executed successfully. And Time complexity has categorised into three ways as Best, average and worst.

Binary search

Algorithm:

② Input:- sorted elements and key
output:- position of key
step ①:- start
step ②:- set $low = 0$ and $high = n-1$
step ③:- while $low \leq high$ then do
step ④:- set $mid = (low + high) / 2$
step ⑤:- if $arr[mid] == key$
step ⑥:- then break
step ⑦:- else if $arr[mid] < key$ then
step ⑧:- set $low = mid + 1$
step ⑨:- else $high = mid - 1$
step ⑩:- if $low > high$ then not found
step ⑪:- end.

Program code:

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
int binarySearch(const vector<int>& arr, int target) {  
    int low = 0;    int high = arr.size() - 1;
```

```
    while (low <= high) {        int  
        mid = low + (high - low) / 2;
```

```
        if (arr[mid] == target) {  
            return mid;
```

```
        } else if (arr[mid] < target) {
low = mid + 1;        } else {        high
= mid - 1;
        }    }    return
-1;
}

int main() {    int size;    cout << "Enter
the number of
elements: ";
    cin >> size;

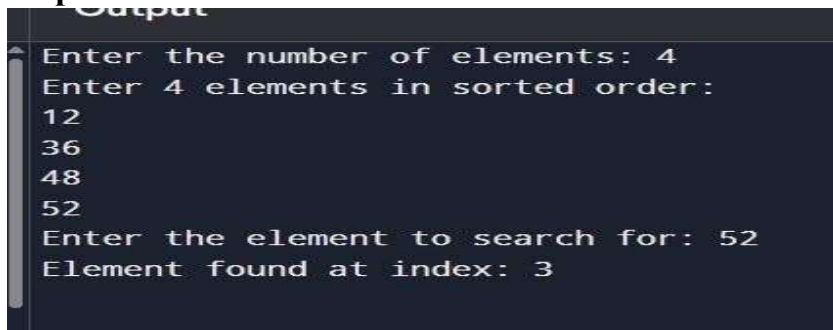
    vector<int> arr(size);    cout << "Enter " << size << "
elements in sorted order:\n";
    for (int i = 0; i < size; i++) {
cin >> arr[i];
    }

    int target;    cout << "Enter the element to
search for: ";
    cin >> target;

    int result = binarySearch(arr, target);

    if (result != -1) {        cout << "Element found at index:
" << result << endl;    } else {        cout << "Element
not found in the array." << endl;
    }

    return 0;
}
```

Output:

```
Output
Enter the number of elements: 4
Enter 4 elements in sorted order:
12
36
48
52
Enter the element to search for: 52
Element found at index: 3
```

Time complexity:

```

→ binary search (A[], n, key) { — T(n)
    low = 0;
    high = n-1; — 1
    while (low <= high) { — (log n)
        mid = (low+high)/2; — T
        if (A[mid] == key) {
            return mid; — (log n)
        }
        else if (key < A[mid]) { — (log n)
            high = mid-1; — (log n)
        }
        else {
            low = mid+1; — (log n)
        }
    }
    return -1;
    T(n) = O(log n)
    
```

Best case	Average case	Worst case
$mid = (low + high) / 2 \rightarrow$ $if (mid == key) \rightarrow$ $T(n) = 1 + 1 = 2$ $T(n) = O(1)$	$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow 1$ no. of divisions $\geq \log_2 n$ $T(n) = \log_2 n$ $T(n) = O(\log n)$	$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots$ $T(n) = \log_2 n$ $T(n) = O(\log n)$

Conclusion:

From this practical of the algorithm and programs have executed successfully. And Time complexity has categorised into three ways as Best, average and worst.